3. The Guardian

TSYP14 City Brain Convergence · Phase 1 · **Triplet 03** · Guardian team "black box" · Report part 3 · Written by the Guardian team · 8 November 2026
Repository folder: `triplet-03/guardian/` · Contract version 1.0 · Key question: **"Is this data telling the truth?"**

7	**9**	**11 / 11**	**113**
trust checks per reading (L1–L6, L3 split in a/b)	attack patterns correlated across readings	Test Book scenarios pass live (logs committed)	automated tests, 97% code coverage

3.1 Role and goals

The Guardian is the trust layer of the triplet. It sits between what the city reports (the **Twin**: sensors and actuators) and what the city does (the **City Brain**: decisions). The Brain cannot act safely on raw readings: data can be faked (a flood that is not there), replayed, frozen by a dead sensor, or drowned in a flood of emergency calls. The Guardian judges every reading and every actuator order, and contains what it finds.

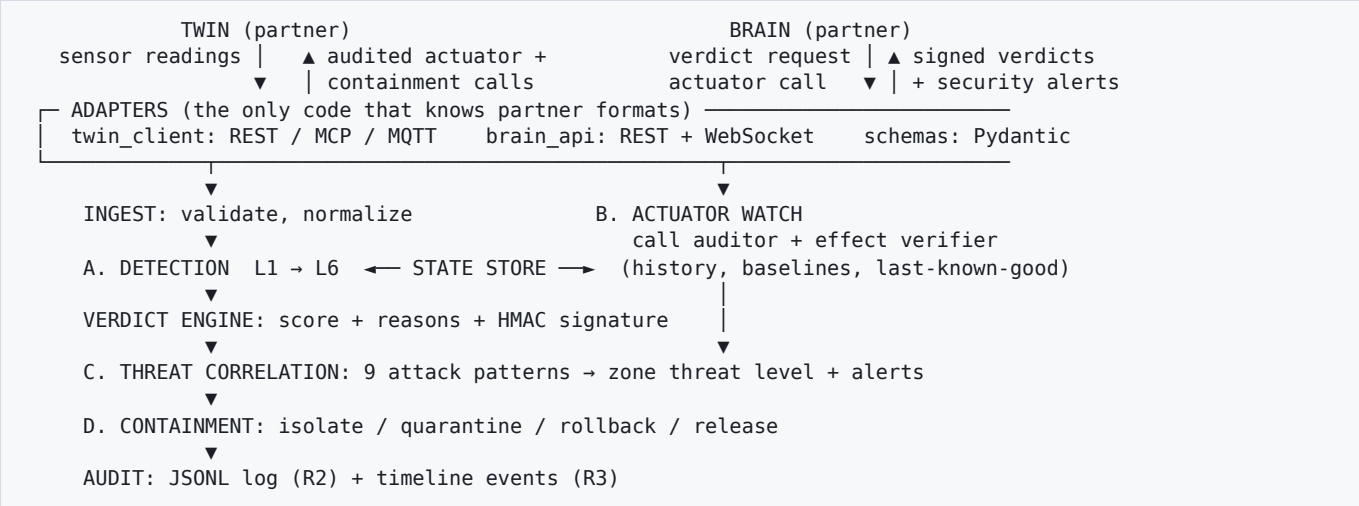
Threats covered

Threat	Detected by	Response
Sensor fault (stuck, dead, out of range, wrong unit)	L3a range/unit, L4 flatline, L6 heartbeat	*suspect* / *unknown*, isolate if persistent
Spoofed / false data (fake flood, impossible event)	L3b city model, L4 dynamics, L5 corroboration	*untrusted*, isolate, roll back
Replay and tampering	L1 HMAC + schema, L2 sequence numbers	*untrusted*, tampering / replay alert
Flooding (calls, messages)	L6 arrival rate, duplicate IDs	Throttle, corroborate, never drop
Actuator failure (ordered ≠ happened)	Effect verifier with an independent sensor	`actuator_failure`, mark unreliable
Cyber-attacks across readings	Threat correlator, 9 patterns (3.2.3)	Zone threat level, alert, quarantine
Unsafe or injected commands	Call auditor: trust_ref, policies, rate limits	Block, quarantine the caller

> **Design principle.** Simple, explainable detectors that visibly catch the attack beat a black-box model. Every verdict names its reasons, so the Brain, the jury and the timeline can see *why* a reading was rejected. Optional ML can only lower trust, never raise it, and is off the critical path.

3.2 Architecture

Four functional blocks share one state store, wrapped by adapters on the outside and an audit trail at the bottom.



Two flows. *Reading flow* (continuous): Twin → ingest → L1–L6 → verdict → state store, with threat correlation consuming every verdict. *Decision flow* (on demand): the Brain requests a verdict or sends an actuator call → the auditor checks its `trust_ref` and safety policies → forward or block → the effect verifier watches independent sensors for the expected physical change.

3.2.1 Block A: the trust checks on every reading

Cheap, definitive checks run first; statistical and cross-sensor checks run last. Failing L1 or L2 makes a reading **untrusted** at once (hard gate). L3–L6 adjust a score that starts at 1.0.

Layer	Question	How it checks	Catches
L1 Authenticity	Genuine and well-formed?	Pydantic schema, `contract_version`, HMAC-SHA256 over a canonical body, constant-time compare	Tampering, injection, malformed data
L2 Freshness	Is it new?	`sequence_no` strictly increasing per sensor, timestamp skew; agreed `sequence_reset` flag after a Twin restart	Replay attacks
L3a Range & unit	Physically possible?	Per-type ranges, internal consistency (load ≤ capacity), unit heuristic (a value ~100× over range that fits after ÷100 = cm sent as m)	Faulty values, unit mix-ups
L3b Semantic	Possible *here*?	City model in `city_model.yaml`: fire in open sea, flood above a zone's maximum level, sensor in a zone without that feature	Impossible context, crude spoofing
L4 Dynamics	Does the change make sense?	Rate of change, flatline (N identical readings), EWMA / z-score, per-hour baseline, slow same-sign drift	Fake spikes, frozen sensors, drift
L5 Corroboration	Does the city agree?	Independent sources from `crosschecks.yaml` (rain, flow, upstream level for water; neighbouring zones), minimum source count	Single-sensor and subtle spoofing
L6 Liveness / rate	Alive, normal pace?	No reading within 2× the expected interval → stale; > 5 emergency calls/zone/min → flood; duplicate `call_id`	Dead devices, call floods

Scoring (`thresholds.yaml`): plausibility fail −0.5, semantic suspect −0.3, dynamics fail −0.3, corroboration fail −0.5, corroboration pass +0.2, liveness fail −0.3, pattern fail −0.2. Cut-offs: ≥ 0.8 **trusted**, 0.4–0.8 **suspect**, < 0.4 **untrusted**; stale or missing data is **unknown**. In a zone whose threat level is raised, *trusted* needs ≥ 0.9 and two independent corroborating sources. Notable readings (big moves) need corroboration to reach *trusted*.

Example (scenario T1, from the committed log): the port sensor `wl-port-03` reports 1.80 m while the real level is 0.40 m and there is no rain. Within 10 s the verdict is **untrusted** with reasons `rate_of_change, no_rainfall_corroboration, no_flow_corroboration, no_upstream_level_corroboration`; the sensor is isolated; `/v1/readings/wl-port-03/latest` serves the rolled-back 0.41 m; the Brain mock ignores the reading and does not move the valve.

3.2.2 Block B: actuator watch (ordered vs. happened)

The Brain sends every actuator call *through* the Guardian (`POST /v1/actuator-calls`). A Twin reply `status: ok` only means "accepted"; a stuck valve or a lying controller can answer ok and do nothing.

Check	Detail
1. Authorisation	The call carries a `trust_ref` to a verdict that is *trusted* and not expired (TTL 30 s). No trust_ref, or a quarantined target → blocked.
2. Policy / safety invariants	From `policies.yaml`: e.g. no grid disconnect on the hospital zone, no duplicate dispatch for one `incident_id`, per-caller rate limits.
3. Effect verification	Expectation with deadline and tolerance, observed by an *independent* sensor: valve closed → flow drops ≥ 80% within 30 s; grid disconnected → zone load ≈ 0; signal green → traffic changes. No effect → `actuator_failure` alert and the actuator is marked unreliable so the Brain picks a backup.
4. Audit	Every call, decision and justification is logged and shown on the timeline.

If the verifying sensor is itself not trusted, the result is **unknown**: the Guardian never accuses an actuator on bad evidence (scenario T5).

3.2.3 Block C: threat correlation

Single-reading checks cannot see attacks spread over many readings. The correlator groups verdicts and calls by zone, device, source and time window against the catalogue in `attack_patterns.yaml`, raises a zone threat level *normal → elevated → under_attack* (which tightens the trust bar), and emits a security alert with type, evidence and suggested response. Levels decay with hysteresis after a quiet period (120 s).

Pattern	Signal	Response
replay	Repeated `sequence_no` with a valid HMAC	Untrusted, flag source
spoofing	Failed corroboration / impossible dynamics	Isolate, roll back
coordinated_spoofing	Several sensors in one zone fail within a window	Zone *under_attack*, stricter trust
flood_dos	Message-rate spike, emergency-call flood	Throttle, mark for corroboration (never drop)
tampering	HMAC failures, schema violations	Reject, possible key compromise alert
command_injection	Calls without trust_ref, bursts	Block; quarantine caller after 3
slow_drift	Sub-threshold same-sign deviation over a long window	Alert, freeze baseline
compromised_twin	Signatures valid but corroboration collapses city-wide	Alert Brain to fall back to safe mode
attack_during_incident	Real corroborated event + fake alarm elsewhere	Keep acting on trusted data only

3.2.4 Block D: containment

Action	Effect	Guardrail
isolate_sensor	Readings no longer usable by the Brain	Cap per zone; evidence logged
quarantine_device	Sensor, actuator or caller blocked	Reversible, reason required
rollback	Serve last-known-good value, marked `rolled_back: true`	Visible in log and API
clear_quarantine	Release after clean readings	Hysteresis, growing cooldown, manual release endpoint

Containment is sent to the Twin as tool calls with `requesting_layer: "guardian"` (the Guardian self-authorises containment). Caps and release paths stop an attacker from using our quarantine against healthy sensors.

3.3 Tech stack and why

Concern	Choice	Why
Language	Python 3.11+, asyncio	Fast to write detection logic; non-blocking pipeline

Brain-facing API	FastAPI (REST + WebSocket), OpenAPI at `/docs`	Standard, self-documenting, validated input
Twin transport	REST push/pull by default; MCP and MQTT adapters	The Twin may expose MCP-style tools or stream; switch by config
Validation	Pydantic v2	Strict schemas and precise error messages
Crypto	stdlib `hmac` / `hashlib` (SHA-256)	No custom crypto; constant-time compare
State	In-memory ring buffers + SQLite	No external infrastructure; survives restart
Statistics / ML	numpy; optional scikit-learn Isolation Forest (off)	Light and explainable; ML can only lower trust
Config	7 YAML files + `.env` for secrets	Nothing partner-specific hard-coded
Logs / timeline	structlog, JSONL; WebSocket; dependency-free HTML screen	Machine-checkable R2 logs, live R3 view
Quality	pytest, pytest-asyncio, hypothesis, ruff, mypy, pre-commit	Unit, integration, fuzz and scenario tests; clean lint and types
Packaging	Docker + docker-compose (Guardian + both mocks)	One command; nothing installed on stage; runs offline

Deliberately avoided: Kafka, Kubernetes, deep learning, external databases. They add demo risk and setup time without improving trust decisions.

3.4 Interfaces in / out

Full contract with an example of every message: `guardian/INTERFACE.md` (Interface Card, R1).

Direction	Message / transport	Purpose
Twin → Guardian	Signed sensor envelope; `POST /v1/readings` (push) or pull / MCP `get_readings` / MQTT `city/readings/#`	The evidence to judge
Brain → Guardian	`POST /v1/verdicts/request`	The gate before acting
Guardian → Brain	Signed verdict; `WS /v1/ws/verdicts` (verdicts + alerts)	Safe decisions
Brain → Guardian → Twin	`POST /v1/actuator-calls` with `trust_ref`, forwarded to `POST {twin}/tools/{action}`	Audited actuator gateway
Guardian → Twin	Containment tool calls (isolate, quarantine, clear)	The city really changes
Guardian → everyone	`WS /v1/ws/timeline`, `/timeline` screen, JSONL logs	Cause and effect (R2, R3)

```
Verdict (Guardian → Brain, abridged)
{ "verdict_id": "verdict-88213",
  "reading_ref": {"sensor_id": "wl-port-03", "sequence_no": 4821},
  "status": "untrusted", "trust_score": 0.2,
  "reasons": ["rate_of_change", "no_rainfall_corroboration", "no_flow_corroboration"],
  "layer_results": [ {"layer": "L1_authenticity", "passed": true, ...}, ... ],
  "action_taken": "isolate_sensor", "threat_level": "normal",
  "ts": "...", "expires_at": "...", "signature": "<HMAC-SHA256>" }
```

Other endpoints: `/health`, `/v1/readings/{id}/latest` (rolled-back value when contained), `/v1/sensors/{id}/status`, `/v1/threat`, `/v1/alerts`, `/v1/containment` (+ `/clear`, `/quarantine`), `/v1/verify/signature`.

Repointing to another partner (Phase 2): change `twin.base_url`, `twin.transport` (rest | mcp | mqtt), `twin.mode` (push | pull) in `config/guardian.yaml` or the env overrides, and the shared keys in `.env`. Inside the triplet compose the Guardian also accepts the shared names `GUARDIAN_URL` (listen port, 8003), `TWIN_URL` and `TWIN_HMAC_SECRET`. No code change.

Input validation and errors: every endpoint answers malformed input with a 4xx and a uniform body `{"error", "message", "field"}`; codes include `schema_invalid`, `contract_version_mismatch`, `hmac_invalid`, `auth_required`, `trust_ref_missing`, `verdict_not_found`. A well-formed reading with a bad HMAC is not an error but an *untrusted* verdict. Bad config files stop startup naming the file and field.

3.5 Security measures

- **Authenticated API:** one API key per partner (`X-API-Key`; `?api_key=` for browser WebSockets, bad key → close 4401). The key decides the caller identity used in audit and quarantine.
- **Signed data both ways:** sensor envelopes verified with the Twin's HMAC key; verdicts signed with the Guardian's key so the Brain can detect a forged verdict. Same canonicalisation rule for both, helper endpoint to check it.
- **Replay protection:** strict per-sensor sequence numbers plus timestamp skew; explicit, signed `sequence_reset` for legitimate restarts.
- **Gate on actions:** every actuator call needs a fresh *trusted* verdict and passes safety invariants and rate limits; callers that inject commands are quarantined.
- **Fail-safe rule:** if the Guardian is unreachable or a verdict is missing, expired or not trusted, the Brain treats data as *unknown* and does not act. Internal errors produce *unknown*, never *trusted*. A silent Twin turns every sensor *unknown* after 2× its interval (scenario T7).
- **Proportionate response:** isolation caps per zone, hysteresis, growing cooldown and a manual release path, so the defence cannot be turned against healthy sensors.
- **Secrets out of code:** keys only in `.env` (git-ignored); the service refuses to start without them.

Handling of false alarms

Risk	How the Guardian avoids it
A real event looks abrupt (burst pipe, real flood)	Dynamics alone only gives *suspect*; corroboration decides. Real flood in T11 stays *trusted*.
Incomplete city map	Semantic failures are *suspect* unless physically impossible (fire in the open sea).
A real disaster produces many calls	Call floods are throttled and marked for corroboration, never dropped.
Our own order changes a sensor (valve closed → flow drops)	Changes explained by an order the Guardian forwarded are not flagged during the expectation window.
Bad verification sensor	An untrusted verify sensor gives *unknown*; the actuator is never accused on bad evidence.
Twin restart resets counters	Signed `sequence_reset` flag is accepted instead of raising replay.
Flapping isolation	Release after clean readings, hysteresis, growing cooldown, isolation caps per zone.

Every decision is logged with its reasons, so thresholds can be tuned on the real Twin during integration.

3.6 Known limitations

- **Compromised Twin:** if the Twin itself lies, HMAC does not help. Corroboration is the defence and weakens as more sensors are spoofed together; city-wide collapse raises `compromised_twin` (safe mode) but cannot say which value is true.
- **HMAC proves origin, not truth:** a compromised sensor signs lies perfectly; that is why L3–L6 exist.
- **Two independent sources** are needed for *trusted* on notable readings; a sensor with no corroboration path can only be *suspect*.
- **Thresholds trade false alarms against missed attacks;** defaults suit the mock city and need tuning on the real Twin.
- **Baselines** learn online from trusted readings only (≥ 200 samples per hour bucket); an attack present from the very start is caught only by the other layers. A recorded normal day from the Twin would help.
- **Incomplete city model** causes semantic false alarms, so those stay *suspect* unless impossible.
- **Stolen Brain key** could be used to get a caller quarantined.
- **Single process:** state in memory + SQLite; several replicas would need shared state. Effect windows are fixed per action.